Agentic LLM Systems:
Reverse and Forward Engineering

Лекция 3

Анатомия агента

Алексеев Илья

DeepPavlov

Задача исправления файла

Требуемый результат

Найти ошибку → исправить файл → проверить

Ответ модели

Предложено исправление:

{"max_retries": 3}

?

config.json · состояние в среде

Исходная ошибка сохраняется:

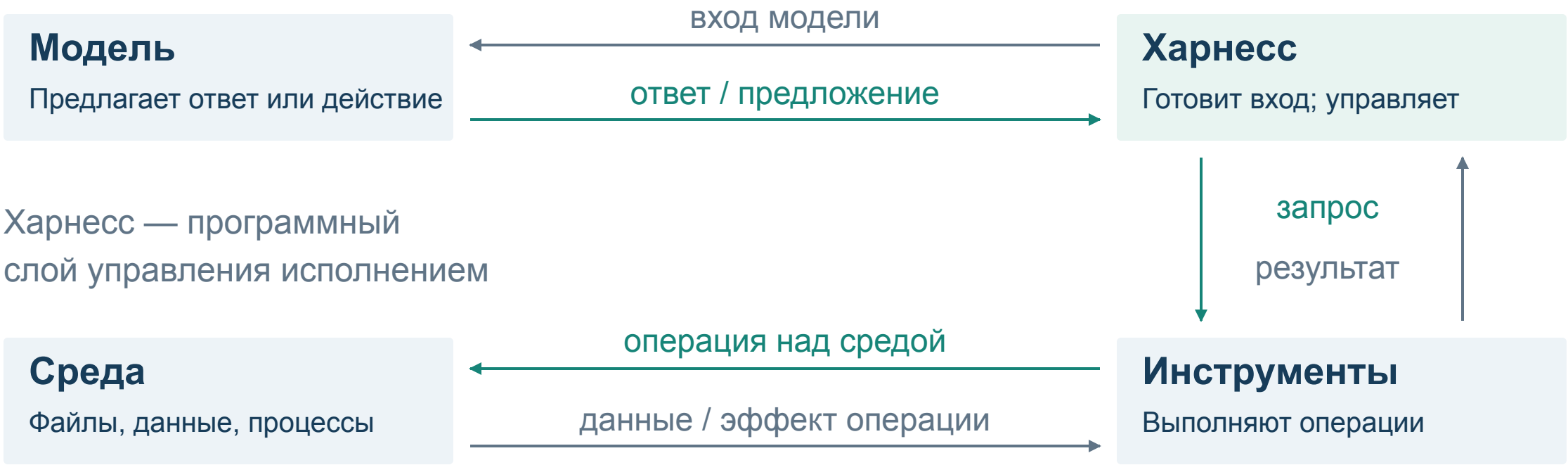
{"max_retries": "3"}

Запись не подтверждена

Отчёта проверки пока нет

Результат задачи: изменённый файл и проверка этой версии.

Компоненты агентной системы



Модель предлагает действие; харнесс организует исполнение.

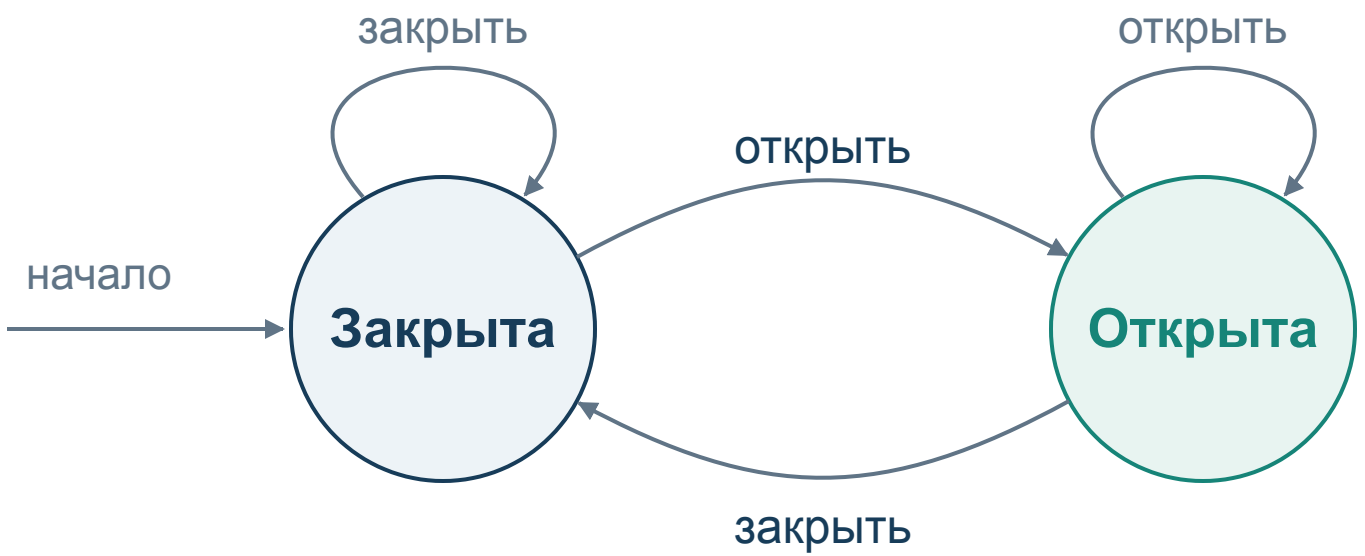
Конечный автомат

Состояние фиксирует текущий режим работы системы.

- Конечное множество состояний и начальное состояние.
- Входные события, на которые реагирует система.
- Правило перехода: текущее состояние и событие задают следующее состояние.

Детерминированность: для одной пары «состояние — событие» задан один переход.

Граф состояний



Обозначения

Вершина — состояние

Стрелка — переход

Подпись — событие

Петля — то же состояние

Пример пути в графе

Закрыта — открыть → Открыта — закрыть → Закрыта

Граф задаёт возможные переходы; путь в графе — конкретную последовательность переходов.

Представление диалога и внешнее действие

1. **Представление диалога:** сообщения, шаблоны и сериализация.
2. **Структурированный выход:** контракт и проверка ответа.
3. **Вызов инструментов:** передача действия исполнителю и получение результата.

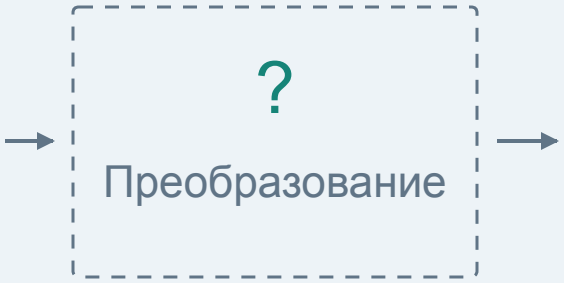
Представление диалога

В приложении

Список сообщений

system
Ответ должен быть кратким.

user
Вычислить 17 × 23.



На входе текстовой модели

Одна последовательность идентификаторов токенов

t1, t2, t3, …, tn

Обозначения условны

Роли и содержимое — поля данных приложения.

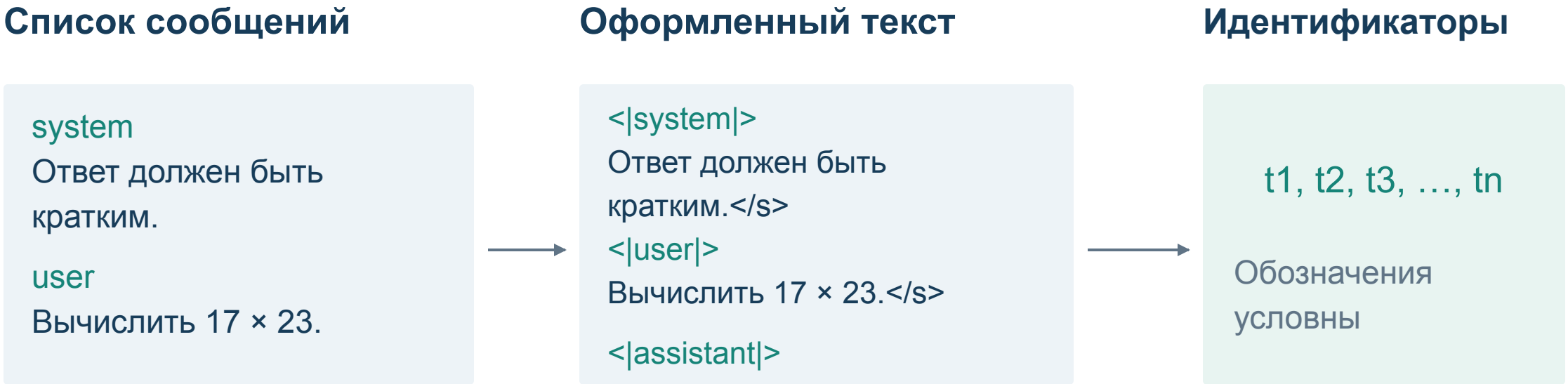
Между сообщениями приложения и входом модели требуется преобразование.

Сообщения и роли

Роль	Содержимое	Назначение
system	Ответ должен быть кратким.	Общие инструкции
user	Вычислить 17 × 23.	Задача или уточнение
assistant	Выход модели	
Результат инструмента	Отдельное сообщение или блок — зависит от API	

Роль обозначает назначение сообщения в протоколе.

Сериализация диалога



Пример оформления: Zephyr-7B-β

Шаблон задаёт текстовый формат, токенизатор преобразует текст в идентификаторы.

Шаблоны и сериализация

Пример: шаблон диалога Zephyr-7B-β.

```
{%- for message in messages -%}
{{ '<|' + message['role'] + '|>\n' + message['content'] + eos_token + '\n' }}
{%- endfor -%}
{%- if add_generation_prompt -%}
{{ '<|assistant|>\n' }}
{%- endif -%}
```

Jinja исполняется программой подготовки входа.

Результат сериализации

```
<|system|>
Ответ должен быть кратким.</s>
<|user|>
Вычислить 17 × 23.</s>
<|assistant|>
```

Последний заголовок задаёт начало нового ответа.

Обозначения ролей и специальные токены

- В Zephyr `</s>` зарегистрирован как специальный токен.
- `<|user|>` не зарегистрирован как отдельный специальный токен.
- Строка, похожая на тег, может состоять из нескольких токенов.
- Остановка генерации относится к одному ответу модели.

Структурированный выход

Исходная фраза: «Партия содержит 12 датчиков».

Требуемый результат:

```
{"item": "датчик", "count": 12}
```

Программная обработка требует контракта полей и значений.

Контракт ответа

- `item`: строка.
- `count`: целое число, не меньше нуля.
- Оба поля обязательны.
- Дополнительные поля запрещены.

JSON Schema задаёт проверяемые ограничения структуры.

Контракт в JSON Schema

```
{
  "type": "object",
  "properties": {
    "item": {"type": "string"},
    "count": {"type": "integer", "minimum": 0}
  },
  "required": ["item", "count"],
  "additionalProperties": false
}
```

`properties` — поля и ограничения; `required` — обязательные поля.

Уровни корректности ответа

Исходные данные: «Партия содержит 12 датчиков»

Контракт: item — строка; count — целое число ≥ 0

1 · Синтаксис

{"item": "датчик",
"count": 12,}

JSON не разбирается

Первая ошибка:

Лишняя запятая

2 · Структура

{"item": "датчик",
"count": "12"}

JSON разобран

Первая ошибка:

Строка вместо числа

3 · Содержание

{"item": "датчик",
"count": 120}

JSON и схема корректны

Первая ошибка:

Количество искажено

Соответствие схеме ещё не подтверждает правильность значений.

Обеспечение формата ответа

ДО ГЕНЕРАЦИИ	ВО ВРЕМЯ	ПОСЛЕ	ПОСЛЕ СТРУКТУРЫ
Инструкция	**Ограничение**	**Валидация**	**Проверка смысла**
«Верни JSON с полем count»	Допустимые продолжения	Полнота → разбор → схема	count совпадает с данными?
Задаёт требование в контексте	Грамматика сужает выбор токенов	Проверяет уже полученный ответ	Отдельный критерий задачи

Инструкция влияет на ответ; грамматика ограничивает форму; проверки выявляют нарушения.

Инструкция, ограничение генерации и валидация дополняют друг друга.

Результат проверки контракта

- Корректное значение допускает дальнейшую обработку.
- Нарушение требует исправленного ответа, уточнения или остановки.
- Решение о продолжении принимает харнесс.

Следующий случай: структурированное значение описывает действие.

Вызов инструментов

Структурированное описание действия требует исполнителя и возврата результата.

Предложение действия: вычислить `17 × 23` инструментом `multiply`.

- Требуется определить операцию и аргументы.
- Требуется вызвать исполнителя.
- Требуется вернуть результат в контекст модели.

Описание инструмента

КОНТЕКСТ МОДЕЛИ

multiply

Произведение двух чисел

a: number b: number

Оба поля обязательны

Реестр

инструментов

имя + аргументы

СТОРОНА ИСПОЛНИТЕЛЯ

multiply(a, b)

Вычислить a × b

Вернуть числовой результат

multiply(17, 23) → 391

Описание и реализация согласованы по имени и контракту аргументов

Описание задаёт интерфейс операции; реализация находится у исполнителя.

Предложение вызова

```
{
  "kind": "tool_call",
  "call_id": "call_01",
  "name": "multiply",
  "arguments": {"a": 17, "b": 23}
}
```

Учебное внутреннее представление харнесса после разбора ответа.

Проверка вызова перед исполнением

ПРЕДЛОЖЕННЫЙ ВЫЗОВ

ИСПОЛНЕНИЕ ПОСЛЕ ВСЕХ ПРОВЕРОК



При необходимости: согласование человека до исполнения.

Исполнение допускается после проверки вызова и необходимых согласований.

Исполнение и результат

```
{
  "kind": "tool_result",
  "call_id": "call_01",
  "status": "success",
  "result": 391
}
```

Харнесс добавляет вызов и связанный результат в историю.

API и формат модели

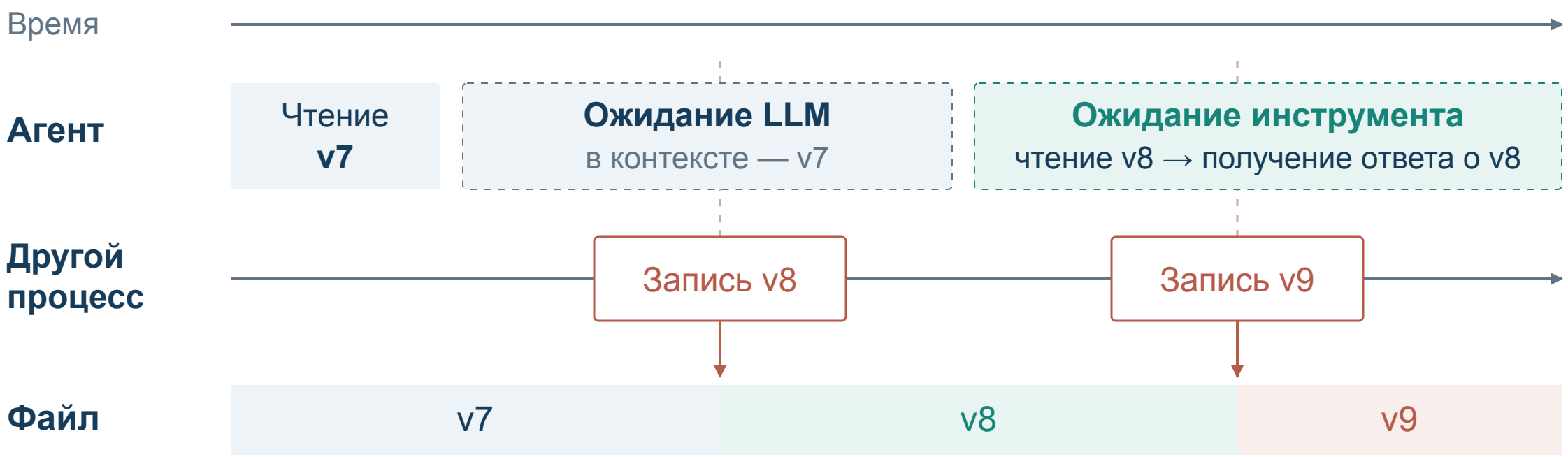
ВХОД: ОТ ОБЪЕКТОВ К ПОСЛЕДОВАТЕЛЬНОСТИ



Схема преобразований условная: поля API и формат последовательности зависят от системы.

Поля API и фактическая последовательность для модели относятся к разным представлениям.

Наблюдение и состояние среды



Ответ о v8 получен, но текущая версия файла — v9.

За время ожидания LLM или инструмента состояние среды может измениться.

Обработка одного внешнего действия



Один проход: от подготовленного входа до зарегистрированного результата действия.

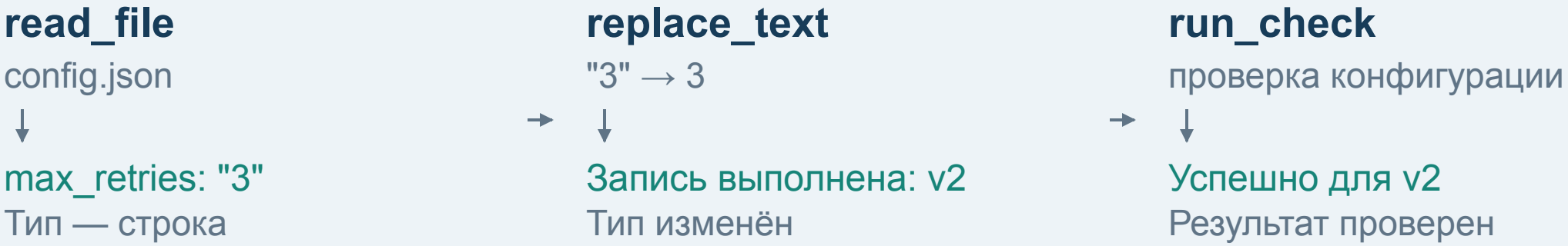
Полученное наблюдение становится основанием для следующего действия.

Организация выполнения задачи

1. **ReAct:** выбор следующего действия по наблюдению.
2. **Управление исполнением:** состояния и данные харнесса.
3. **Участие человека:** уточнение, согласование и ожидание.

ReAct

Вызов инструмента → наблюдение



Учебное пояснение выбора действия · не описание внутренних вычислений модели

Нужно узнать содержимое файла	Строку заменяем целым числом	После записи нужен отчёт о проверке

Наблюдения уточняют основания следующего решения.

Управление исполнением

Один последовательный вызов инструмента



Модель
генерирует ответ

Харнесс
управляет порядком и остальными состояниями

Харнесс организует повторение генерации, исполнения и получения наблюдения.

Данные исполнения

Управляющее состояние

Исполнение инструмента → **Добавление** наблюдения

Переход после результата

1. Сменить состояние
2. Обновить данные

Набор состояний конечен;
история и счётчики изменяются.

Изменяемые данные харнесса

История
+ вызов и его результат

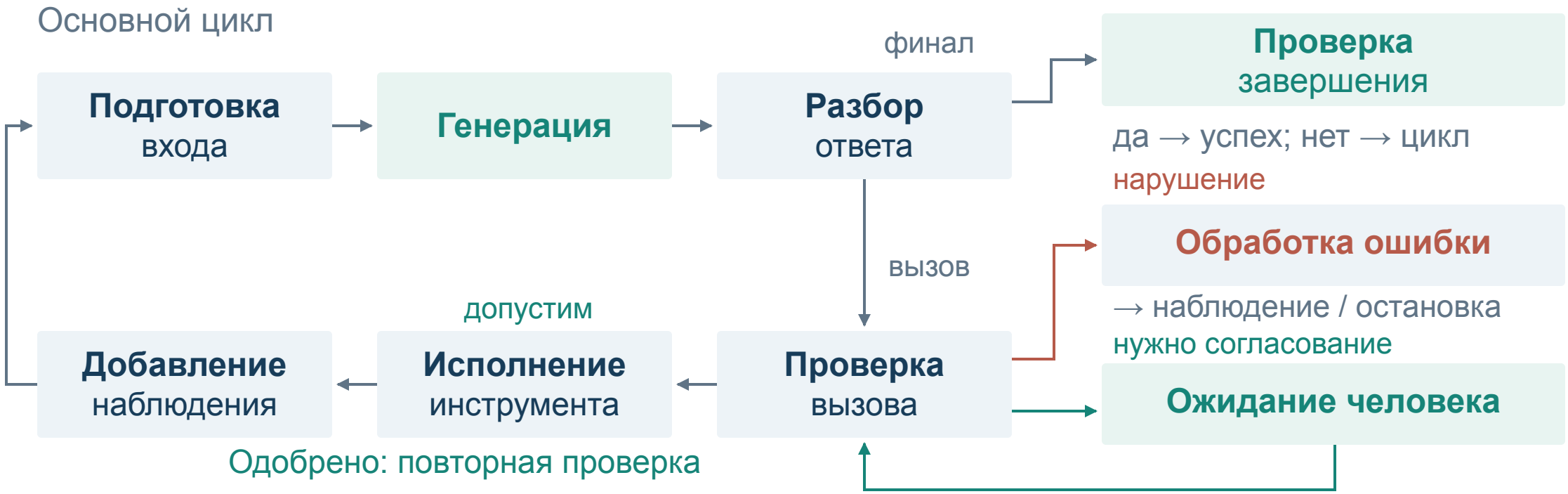
Текущий вызов и согласование
+ статус исполнения

Счётчики и бюджет
+ затраты на операцию

Свидетельства результата
+ отчёт и версия файла

Расширенный конечный автомат объединяет управляющие состояния и изменяемые данные.

Условия переходов

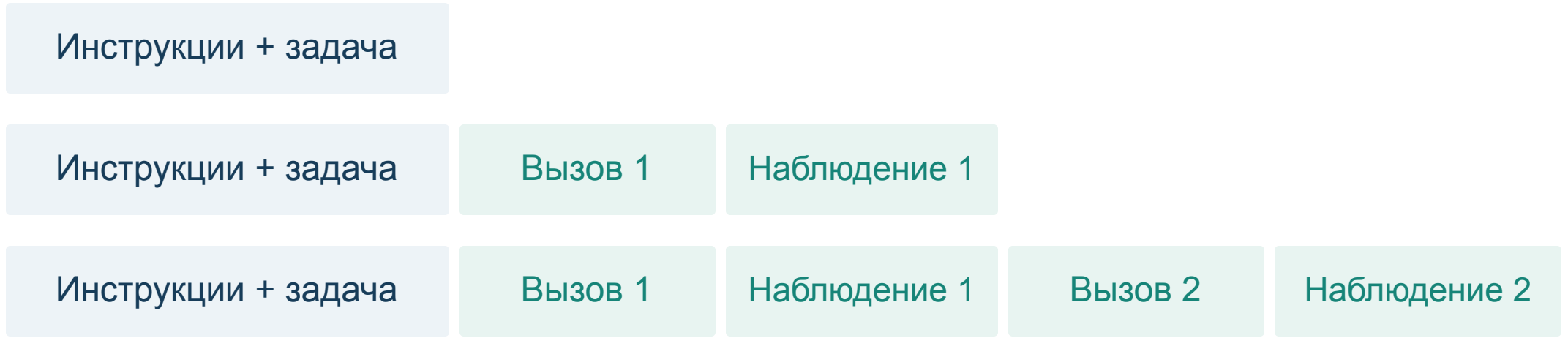


Следующее состояние определяется условиями перехода и текущими данными.

Рост истории

Входы последовательных обращений к модели

Длина полос условная



Общий префикс сохраняется; новые сообщения добавляются справа.

При передаче полной истории вход растёт и содержит повторяющийся префикс.

ReAct и автомат харнесса

ReAct: организация решения через рассуждение, действие и наблюдение.

Автомат: состояния, условия переходов и обновление данных.

Политика исполнения применяется независимо от формулировок модели.

Участие человека

Для продолжения может отсутствовать информация или разрешение.

- Уточнение задачи.
- Согласование действия.
- Передача управления специалисту.

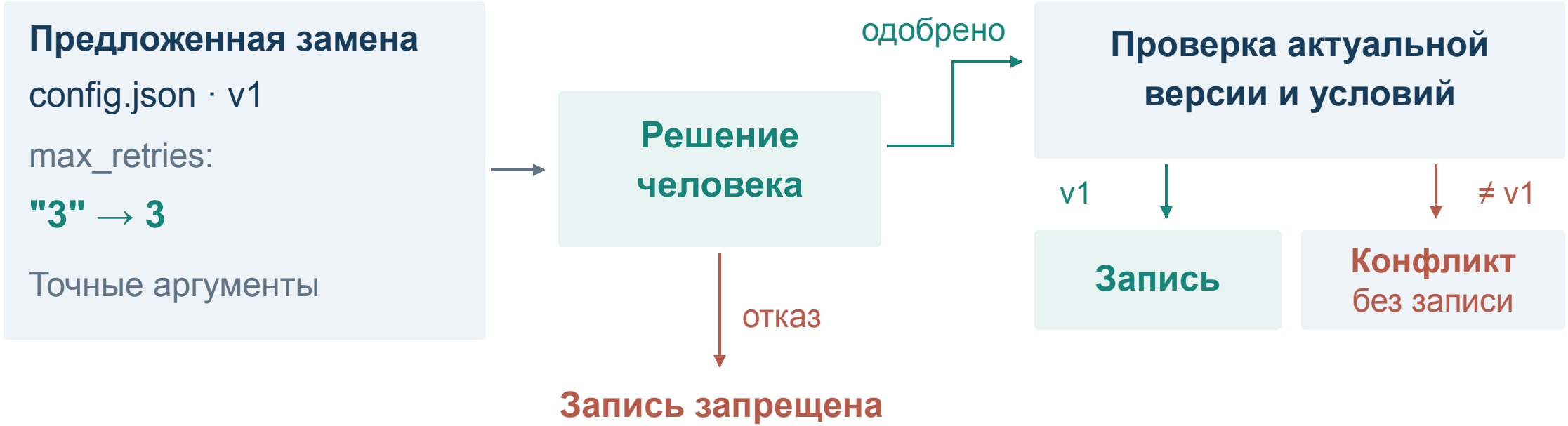
Ожидание человека является штатным состоянием.

Причины обращения к человеку

- Неизвестно требуемое значение параметра: требуется уточнение.
- Запись защищённого файла требует разрешения: требуется согласование.
- Требования противоречивы: требуется решение специалиста.

Ответ человека определяет допустимое продолжение.

Область согласования



Одобрение связано с этой заменой; новая версия требует новой оценки.

Одобрение относится к конкретному действию; предусловия проверяются повторно.

Ожидание и возобновление

- Сохраняются вызов и причина остановки.
- Продолжение зависит от внешнего ответа.
- Отсутствие ответа сохраняет ожидание или приводит к остановке по сроку.
- Молчание не считается одобрением.

Выполнение задачи агентной системой

Выход модели становится действием после разбора, проверки и исполнения.

Новое наблюдение определяет основания следующего решения.

Завершение подтверждается результатом задачи и состоянием операций.